A arquitetura Transformer: tokens, embeddings e atenção

Transformers e Large Language Models

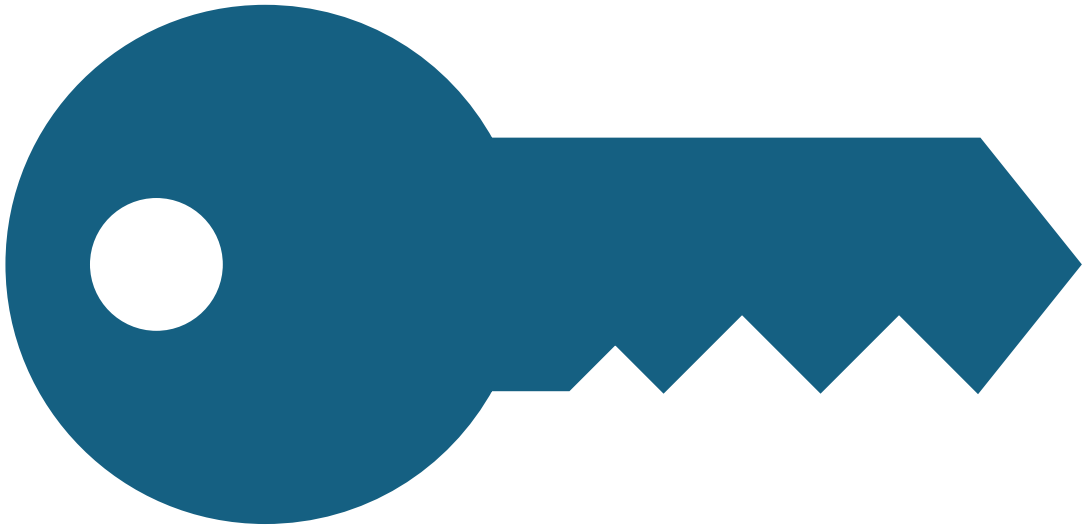
Aula 02 – 31 de Março de 2026

Vítor Wilher

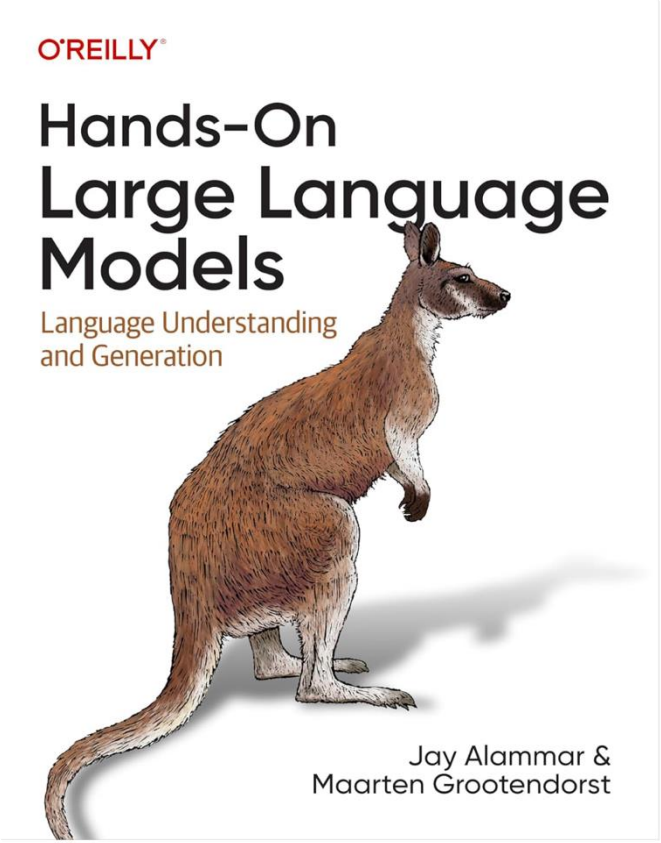
Data Tech Lead na Análise Macro

Plano de Voo

- Dos Modelos de Linguagens aos LLMs
- Dos LLMs aos Modelos de Fundação
- Tokens e Embeddings
- Codificação e Decodificação de Contexto com Atenção
- “Attention is all you need”



Referência das Figuras

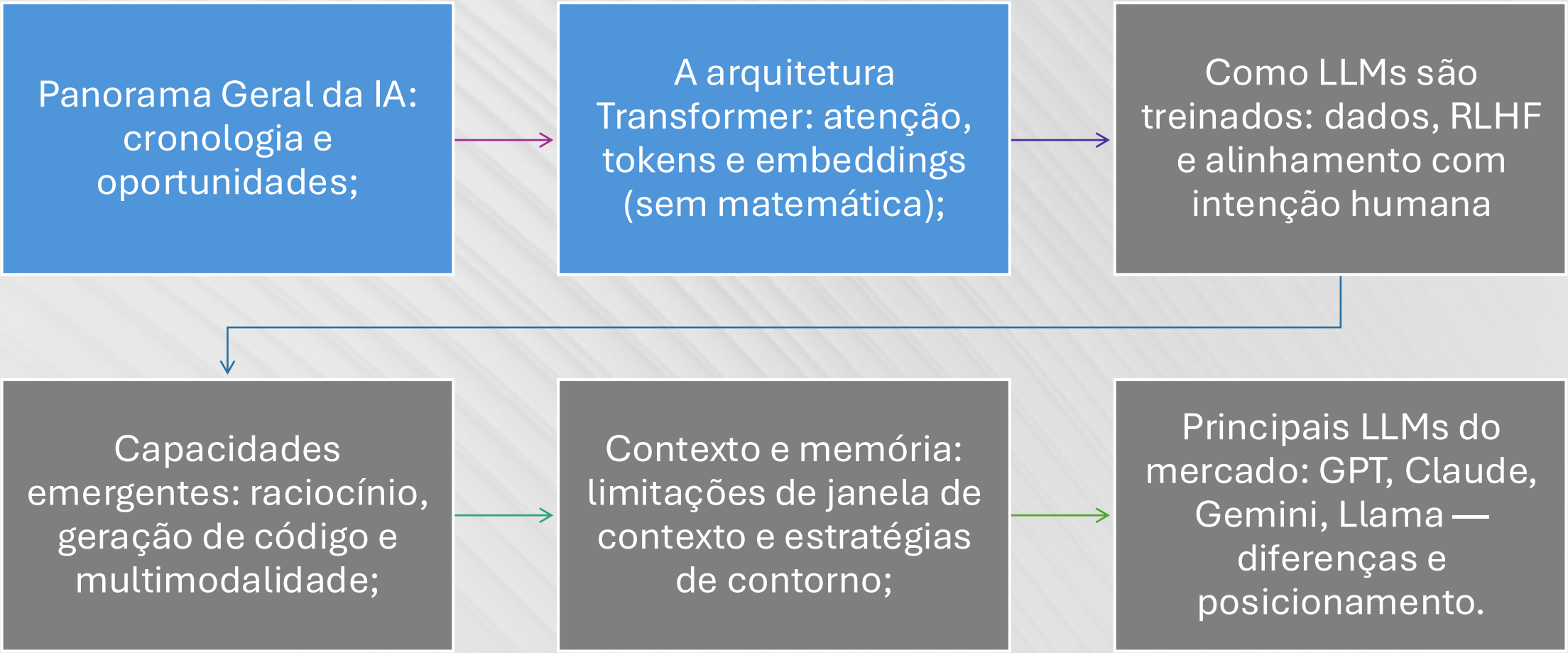


Ementa Resumida do Curso

Detalhar a arquitetura *Transformer* e os *LLMs* — a tecnologia específica que impulsionou a revolução da **IA generativa** em linguagem.

Compreender como esses modelos funcionam é essencial para usá-los estrategicamente.

O que veremos hoje...



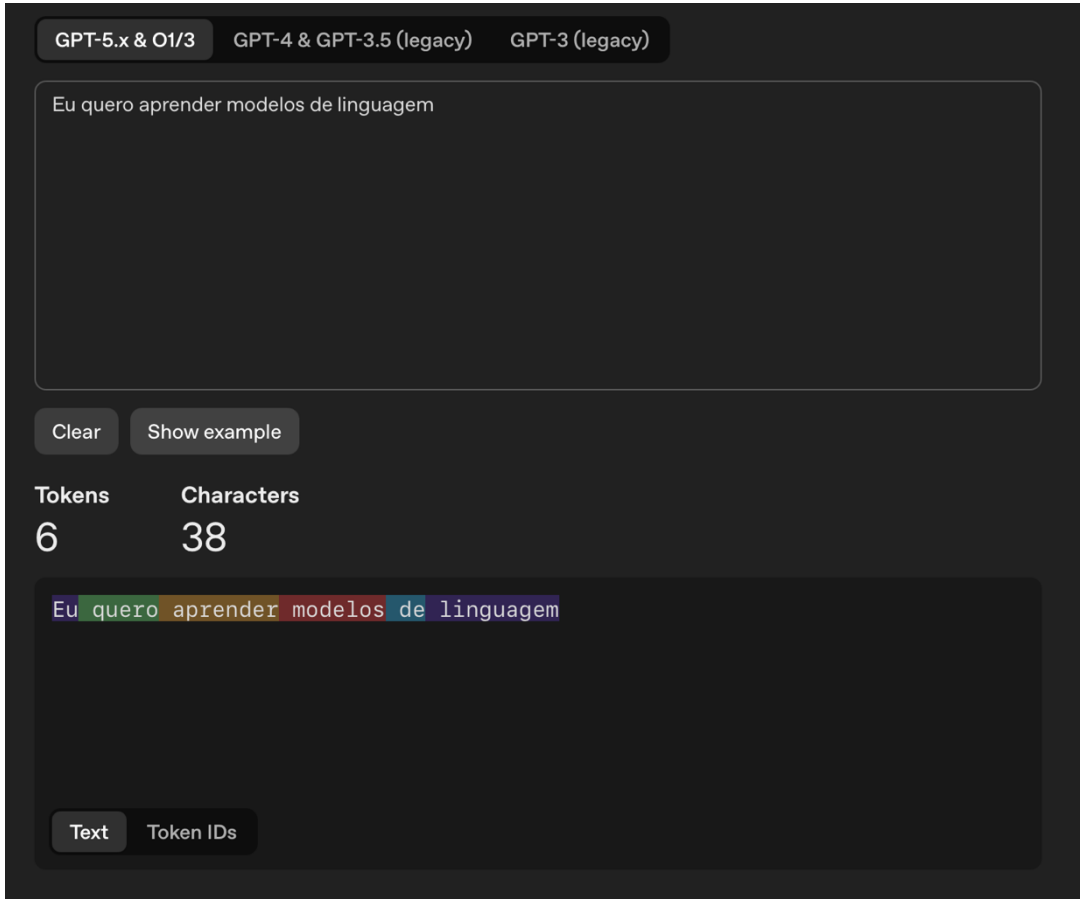
De Modelos de Linguagem a Grandes Modelos de Linguagem

A unidade básica de um modelo de linguagem é *o token*.

Um token pode ser um caractere, uma palavra ou uma parte de uma palavra (como -tion), dependendo do modelo.

O modelo GPT-4, por exemplo, divide a frase abaixo em 9 tokens:

De Modelos de Linguagem a Grandes Modelos de Linguagem



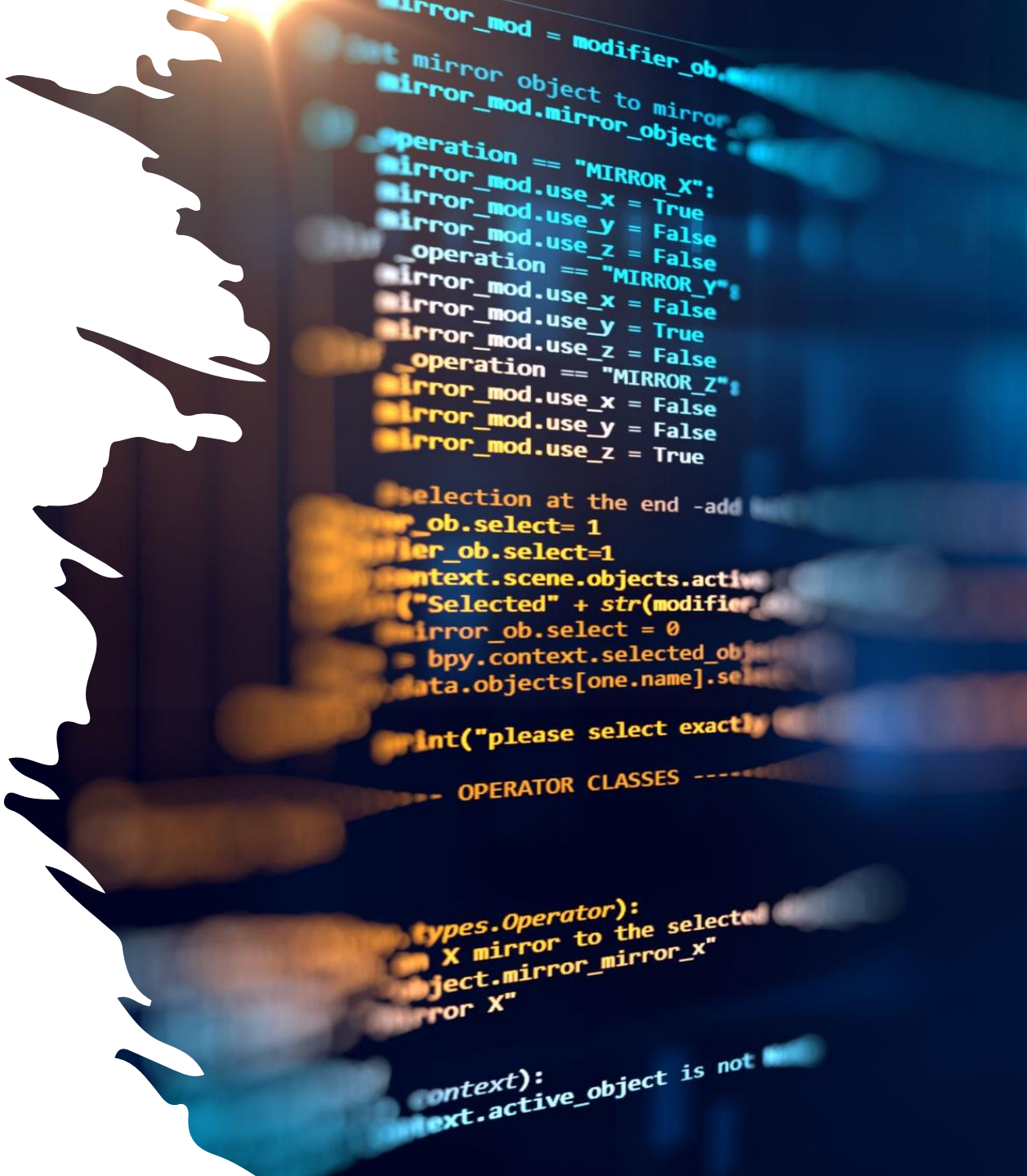
https://platform.openai.com/tokenizer

De Modelos de Linguagem a Grandes Modelos de Linguagem

O processo de dividir o texto original em tokens é chamado *de tokenização*.

Para o GPT-4, um token médio é aproximadamente ¾ do tamanho de uma palavra, por exemplo.

Então, 100 tokens seriam aproximadamente 75 palavras.



De Modelos de Linguagem a Grandes Modelos de Linguagem

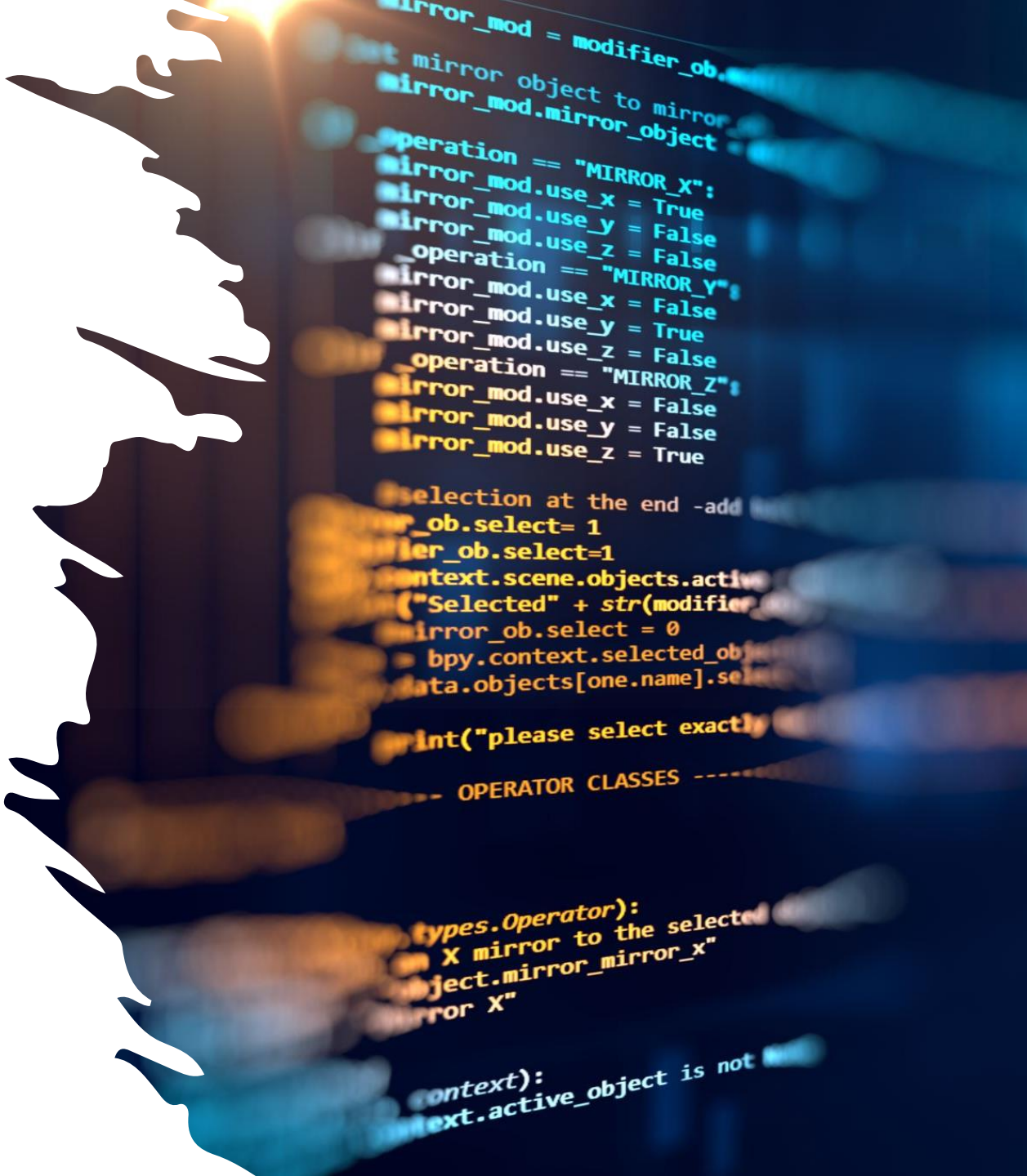
O conjunto de todos os tokens com os que um modelo pode trabalhar é o *vocabulário* do modelo.

Você pode usar um pequeno número de tokens para construir um grande número de palavras distintas, semelhante a como você pode usar algumas letras no alfabeto para construir muitas palavras.

O modelo Mixtral 8x7B tem um tamanho de vocabulário de 32.000.

O tamanho do vocabulário do GPT-4 é 100.256.

O método de tokenização e o tamanho do vocabulário são decididos pelos desenvolvedores do modelo.



Por que os modelos de linguagem usam *token* como sua unidade em vez de *palavra* ou *caractere*?

- Comparados aos caracteres, os tokens permitem que o modelo divida palavras em componentes significativos. Por exemplo, “cooking” pode ser dividido em “cook” e “ing” com ambos os componentes carregando algum significado da palavra original.
- Como há menos tokens exclusivos do que palavras únicas, isso reduz o tamanho do vocabulário do modelo, tornando-o mais eficiente;
- Os tokens também ajudam o modelo a processar palavras desconhecidas. Por exemplo, uma palavra inventada como “chatgpting” poderia ser dividida em “chatgpt” e “ing”, ajudando o modelo a entender sua estrutura. Tokens equilibram com menos unidades do que palavras, mantendo mais significado do que caracteres individuais.

De Modelos de Linguagem a Grandes Modelos de Linguagem

Existem dois tipos principais de modelos de linguagem: *modelos de linguagem mascarados* e *modelos de linguagem autorregressiva*.

Eles diferem com base em quais informações podem usar para prever um *token*.

De Modelos de Linguagem a Grandes Modelos de Linguagem

Um modelo de linguagem mascarada é treinado para prever tokens ausentes em qualquer lugar de uma sequência, *usando o contexto de antes e depois dos tokens ausentes*.

Em essência, um modelo de linguagem mascarada é treinado para ser capaz de preencher o espaço em branco.

Por exemplo, dado o contexto,

“Meu __ favorito é azul”, um modelo de linguagem mascarada deve prever que o espaço em branco é provavelmente “cor”.

Um exemplo bem conhecido de um modelo de linguagem mascarada são as representações bidirecionais de codificadores de transformadores, ou BERT.

De Modelos de Linguagem a Grandes Modelos de Linguagem

Um modelo de linguagem autorregressiva é treinado para prever o próximo token em uma sequência, *usando apenas os tokens anteriores*.

Ele prevê o que vem a seguir em “Minha cor favorita é __.”

Um modelo autorregressivo pode gerar continuamente um token após o outro.

Hoje, os modelos de linguagem autorregressiva são os modelos de escolha para geração de texto e, por esse motivo, são muito mais populares do que os modelos de linguagem mascarada.

De Modelos de Linguagem a Grandes Modelos de Linguagem

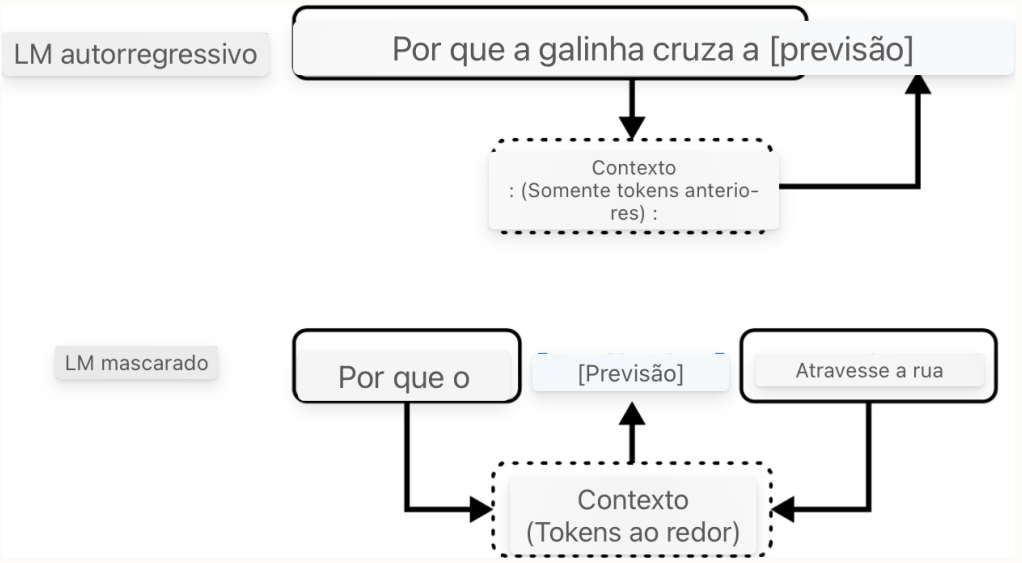


Figura 1-2. Modelo de linguagem autorregressiva e modelo de linguagem mascarada.

De Modelos de Linguagem a Grandes Modelos de Linguagem

- As saídas dos modelos de linguagem são abertas.
- Um modelo de linguagem pode usar seu vocabulário fixo e finito para construir saídas infinitas possíveis.
- Um modelo que pode gerar saídas abertas é chamado *de generativo*, daí o termo *IA generativa*.
- Você pode pensar em um modelo de linguagem como uma *máquina de conclusão*: dado um texto (prompt), ele tenta completar esse texto.
- Aqui está um exemplo:

De Modelos de Linguagem a Grandes Modelos de Linguagem

- As saídas dos modelos de linguagem são abertas.
- Um modelo de linguagem pode usar seu vocabulário fixo e finito para construir saídas infinitas possíveis.
- Um modelo que pode gerar saídas abertas é chamado *de generativo*, daí o termo *IA generativa*.
- Você pode pensar em um modelo de linguagem como uma *máquina de conclusão*: dado um texto (prompt), ele tenta completar esse texto.
- Aqui está um exemplo:

De Modelos de Linguagem a Grandes Modelos de Linguagem

```
Prompt (from user): "To be or not to be"
Completion (from language model): ", that is the question."
```

De Modelos de Linguagem a Grandes Modelos de Linguagem

Por mais simples que pareça, a conclusão é incrivelmente poderosa.

Muitas tarefas, incluindo tradução, resumo, codificação e resolução de problemas matemáticos, podem ser enquadradas como tarefas de conclusão.

Por exemplo, dado o prompt:

“Como você está em francês é ...”,

um modelo de idioma pode ser capaz de completá-lo com:

“Comment ça va”, traduzindo efetivamente de um idioma para outro.

De Modelos de Linguagem a Grandes Modelos de Linguagem

```
Question: Is this email likely spam? Here’s the email: <email c
Answer:
```

Um modelo de linguagem pode ser capaz de completá-lo com: “Spam provavelmente”.

O que transforma esse modelo de idioma em um classificador de spam.

De Modelos de Linguagem a Grandes Modelos de Linguagem

- A modelagem de linguagem é apenas um dos muitos algoritmos de ML.

- Existem também modelos para detecção de objetos, modelagem de tópicos, sistemas de recomendação, previsão do tempo, previsão do preço das ações, etc.

- O que é especial sobre os modelos de linguagem que os tornaram o centro da abordagem de dimensionamento que causou o momento ChatGPT?

De Modelos de Linguagem a Grandes Modelos de Linguagem

- A resposta é que os modelos de linguagem podem ser treinados usando *autosupervisão*, enquanto muitos outros modelos exigem *supervisão*.

- Supervisão refere-se ao processo de treinamento de algoritmos de ML usando dados rotulados, que podem ser caros e lentos para obter.

- A autosupervisão ajuda a superar esse gargalo de rotulagem de dados para criar conjuntos de dados maiores para os modelos aprenderem, permitindo efetivamente que os modelos aumentem.

- Veja como:

De Modelos de Linguagem a Grandes Modelos de Linguagem

- Com supervisão, você rotula exemplos para mostrar os comportamentos que deseja que o modelo aprenda e, em seguida, treina o modelo nesses exemplos.

- Uma vez treinado, o modelo pode ser aplicado a novos dados.

- Por exemplo, para treinar um modelo de detecção de fraude, você usa exemplos de transações, cada uma rotulada como "fraude" ou "não fraude".

- Uma vez que o modelo aprende com esses exemplos, você pode usar esse modelo para prever se uma transação é fraudulenta.

De Modelos de Linguagem a Grandes Modelos de Linguagem

- O sucesso dos modelos de IA na década de 2010 estava na supervisão.
- O modelo que iniciou a revolução do aprendizado profundo, AlexNet (Krizhevsky et al., 2012), foi supervisionado.
- Foi treinado para aprender a classificar mais de 1 milhão de imagens no conjunto de dados ImageNet.
- Ele classificou cada imagem em uma das 1.000 categorias, como “carro”, “balão” ou “macaco”.

De Modelos de Linguagem a Grandes Modelos de Linguagem

- Uma desvantagem da supervisão é que a rotulagem de dados é cara e demorada.

- Se custasse 5 centavos para uma pessoa rotular uma imagem, custaria US$ 50.000 rotular um milhão de imagens para a ImageNet.

- Se você quiser que duas pessoas diferentes rotulem cada imagem - para que você possa verificar a qualidade do rótulo - custaria o dobro.

- Como o mundo contém muito mais de 1.000 objetos, para expandir os recursos dos modelos para trabalhar com mais objetos, você precisaria adicionar rótulos de mais categorias.

- Para escalar até 1 milhão de categorias, o custo de rotulagem sozinho aumentaria para US$ 50 milhões.

De Modelos de Linguagem a Grandes Modelos de Linguagem

- Rotular objetos do cotidiano é algo que a maioria das pessoas pode fazer sem treinamento prévio.
- Portanto, pode ser feito de forma relativamente barata. No entanto, nem todas as tarefas de rotulagem são tão simples.
- Gerar traduções em latim para um modelo de inglês para latim é mais caro.
- Rotular se uma tomografia computadorizada mostra sinais de câncer seria astronômico.

De Modelos de Linguagem a Grandes Modelos de Linguagem

- A autosupervisão ajuda a superar o gargalo de rotulagem de dados.

- Na autosupervisão, em vez de exigir rótulos explícitos, o modelo pode inferir rótulos a partir dos dados de entrada.

- A modelagem de linguagem é autosupervisionada porque cada sequência de entrada fornece tanto os rótulos (tokens a serem previstos) quanto os contextos que o modelo pode usar para prever esses rótulos.

De Modelos de Linguagem a Grandes Modelos de Linguagem

- O aprendizado autosupervisionado significa que os modelos de linguagem podem aprender com sequências de texto sem exigir qualquer rotulagem.

- Como as sequências de texto estão por toda parte - em livros, postagens de blog, artigos e comentários do Reddit - é possível construir uma enorme quantidade de dados de treinamento, permitindo que os modelos de linguagem sejam dimensionados para se tornarem LLMs.

Dos LLMs aos “Modelos de Fundação”

- Embora os modelos de linguagem sejam capazes de tarefas incríveis, eles são limitados ao texto.
- Como humanos, percebemos o mundo não apenas através da linguagem, mas também através da visão, audição, toque e muito mais.
- Ser capaz de processar dados além do texto é essencial para que a IA opere no mundo real.
- Por esse motivo, os modelos de linguagem estão sendo estendidos para incorporar mais modalidades de dados.
- GPT-4V e Claude 3 podem entender imagens e textos.
- Alguns modelos até entendem vídeos, ativos 3D, estruturas de proteínas e assim por diante.
- Incorporar mais modalidades de dados em modelos de linguagem os torna ainda mais poderosos.

Dos LLMs aos “Modelos de Fundação”

- Embora muitas pessoas ainda chamem Gemini e GPT-4V de LLMs, eles são melhor caracterizados como modelos de fundação.

- A palavra fundação significa tanto a importância desses modelos em aplicações de IA quanto o fato de que eles podem ser construídos para diferentes necessidades.

Dos LLMs aos “Modelos de Fundação”

- Os modelos de fundação marcam um avanço na estrutura tradicional da pesquisa de IA.
- Por muito tempo, a pesquisa de IA foi dividida por modalidades de dados.
- O processamento de linguagem natural (PNL) lida apenas com texto.
- A visão computacional lida apenas com visão. Modelos somente de texto podem ser usados para tarefas como tradução e detecção de spam.
- Modelos somente de imagem podem ser usados para detecção de objetos e classificação de imagens.
- Modelos somente de áudio podem lidar com reconhecimento de fala (fala para texto, ou STT) e síntese de fala (texto para fala, ou TTS).

Dos LLMs aos “Modelos de Fundação”

- Um modelo que pode funcionar com mais de uma modalidade de dados também é chamado de modelo multimodal.

- Um modelo multimodal generativo também é chamado de grande modelo multimodal (LMM).

- Se um modelo de linguagem gera o próximo token condicionado a tokens somente de texto, um modelo multimodal gera o próximo token condicionado a tokens de texto e imagem, ou quaisquer modalidades que o modelo suporte.

Dos LLMs aos “Modelos de Fundação”

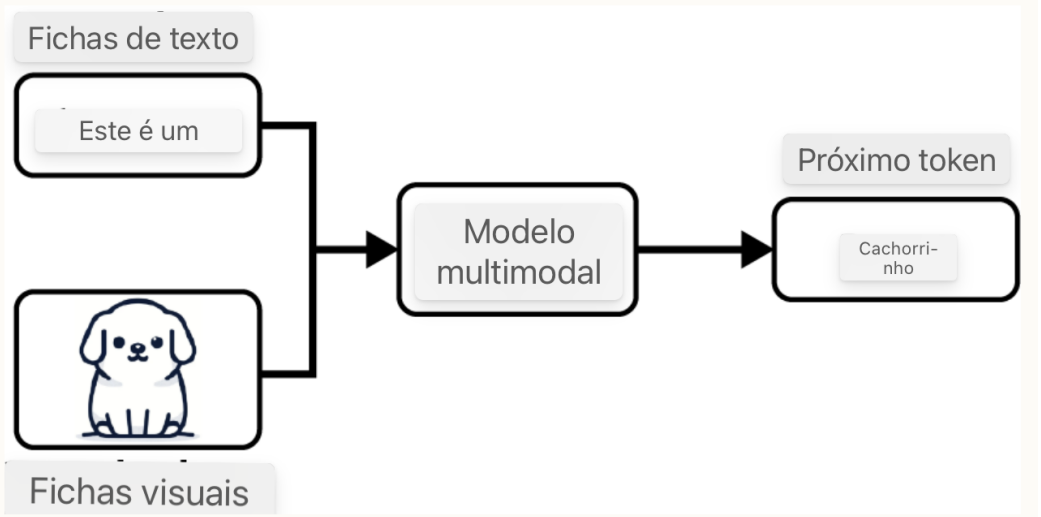


Figura 1-3. Um modelo multimodal pode gerar o próximo token usando informações de tokens de texto e visuais.

Tokens e Embeddings

O saco de palavras funciona da seguinte maneira: vamos supor que temos duas frases para as quais queremos criar representações numéricas.

O primeiro passo do modelo de saco de palavras é *a tokenização*, o processo de dividir as frases em palavras individuais ou subpalavras (*tokens)*.

“Bag of Words”

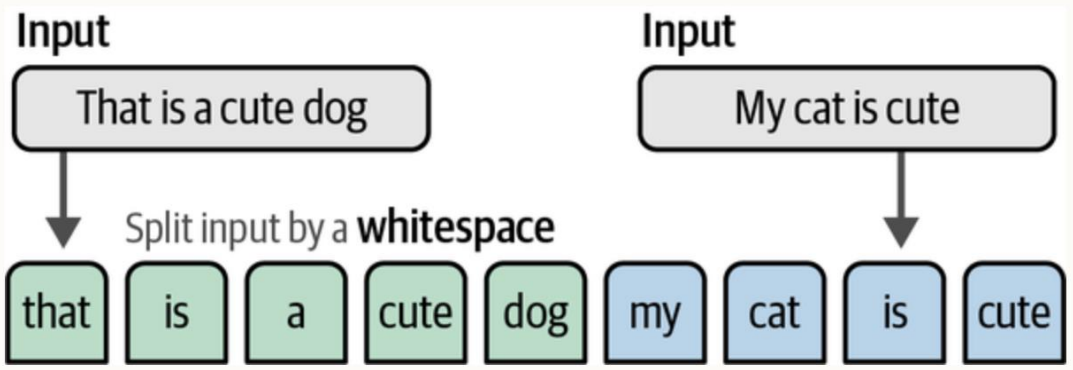


Figure 1-3. Each sentence is split into words (tokens) by splitting on a whitespace.

Tokens e Embeddings

- O método mais comum para tokenização é dividir em um espaço em branco para criar palavras individuais.

- No entanto, isso tem suas desvantagens, pois alguns idiomas, como o mandarim, não têm espaços em branco em torno de palavras individuais.

- Após a tokenização, combinamos todas as palavras únicas de cada frase para criar um **vocabulário** que possamos usar para representar as frases;

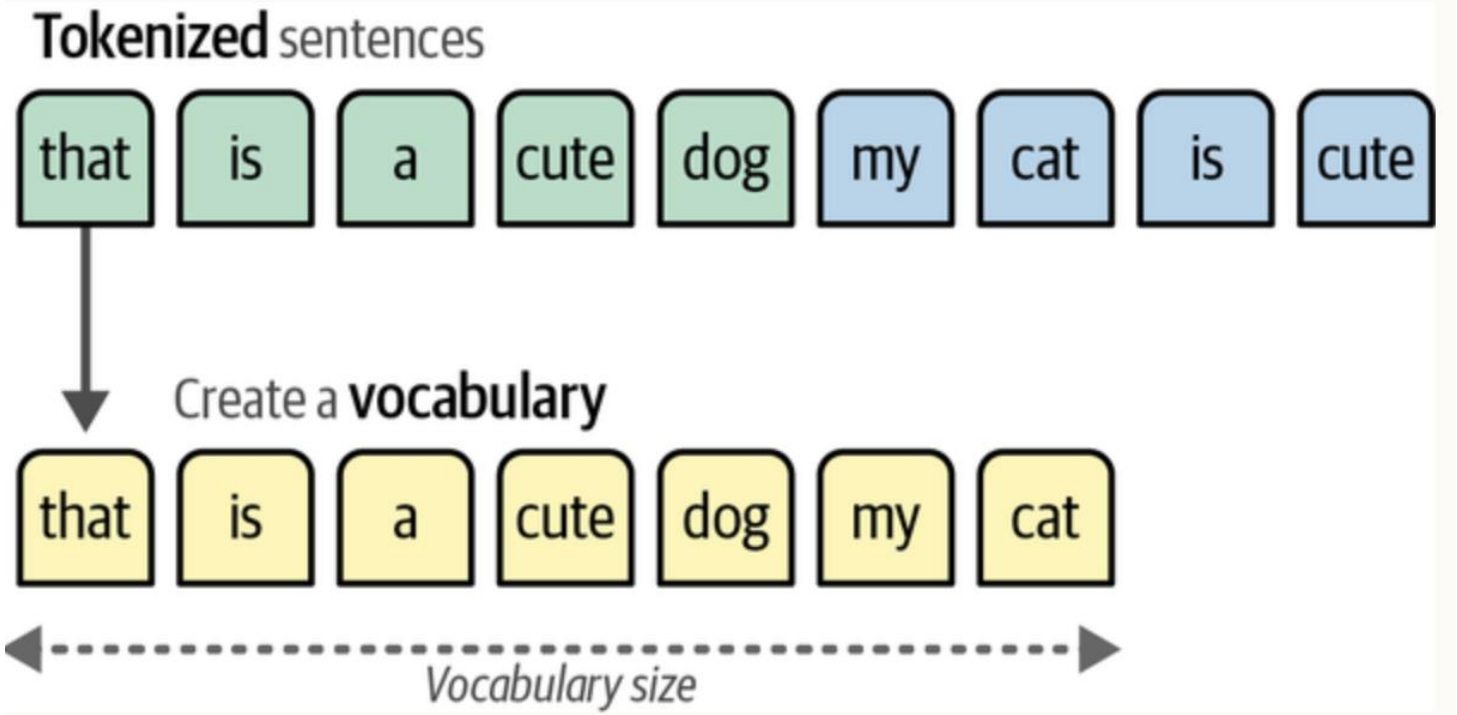
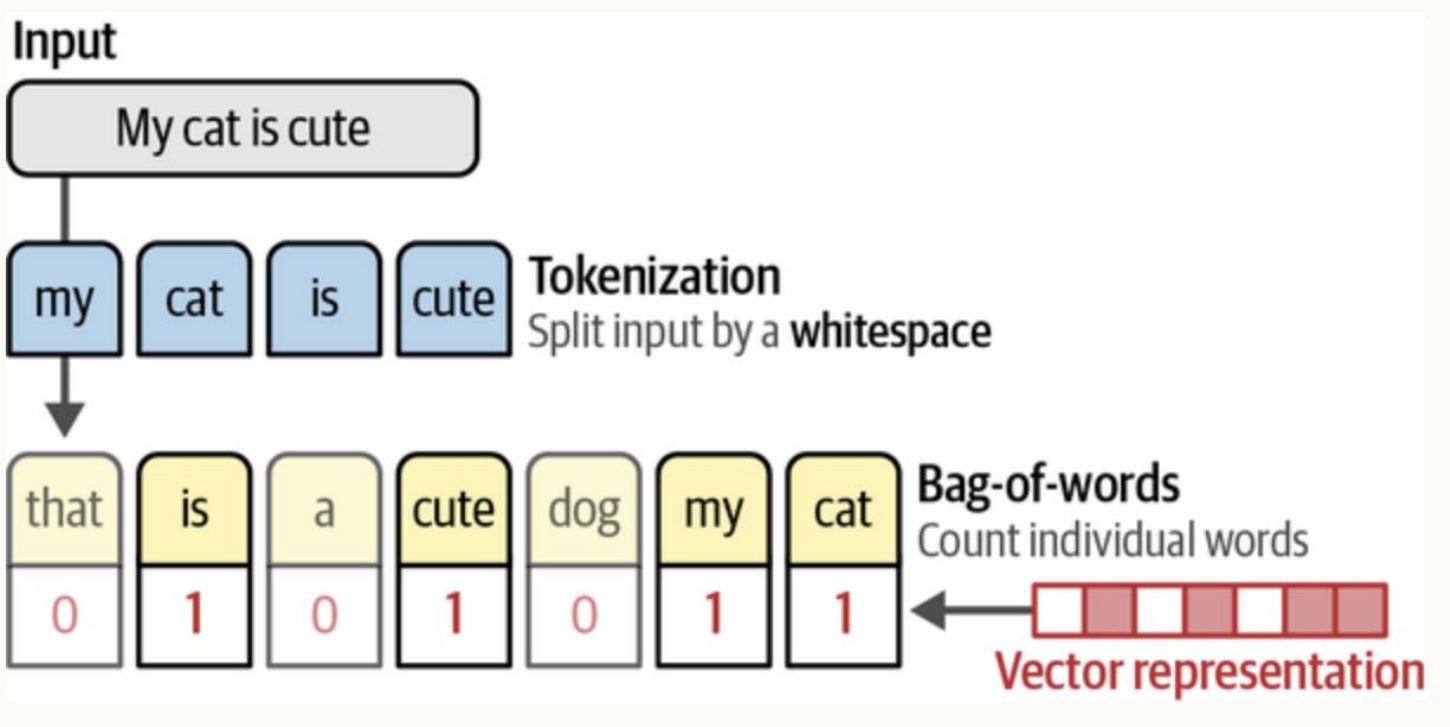


Figure 1-4. A vocabulary is created by retaining all unique words across both sentences.

Tokens e Embeddings

- Usando nosso vocabulário, simplesmente contamos com que frequência uma palavra em cada frase aparece, literalmente criando um saco de palavras. Como resultado, um modelo de saco de palavras visa criar representações de texto na forma de números, também chamadas de vetores ou representações vetoriais.



Tokens e Embeddings

- Saco de palavras, embora uma abordagem elegante, tem uma falha. Considera a linguagem nada mais do que um saco quase literal de palavras e ignora a natureza semântica, ou significado, do texto.

- Lançado em 2013, word2vec foi uma das primeiras tentativas bem-sucedidas de capturar o significado do texto em ***embeddings***.

- Embeddings são representações vetoriais de dados que tentam capturar seu significado.

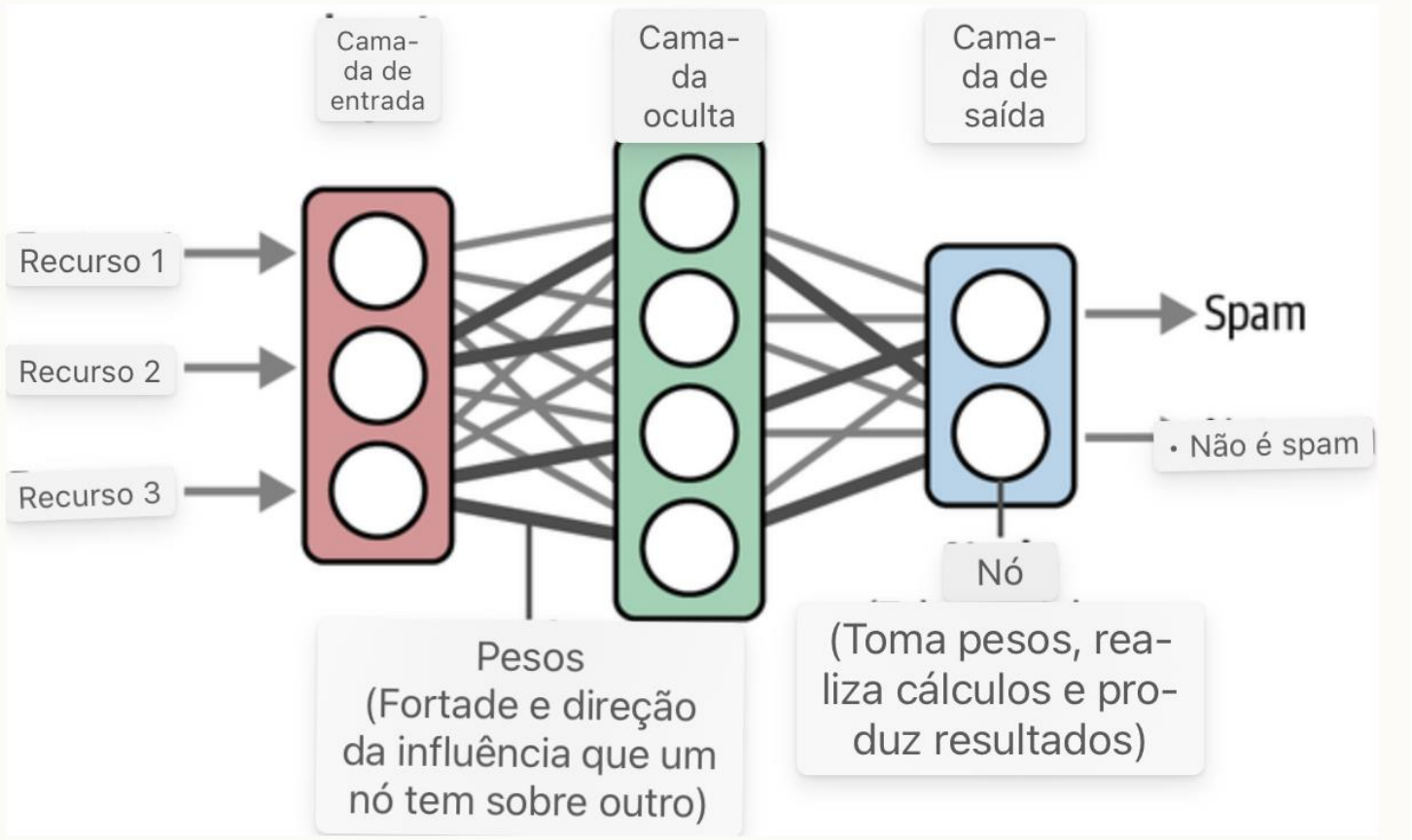
Tokens e Embeddings

- Para gerar essas representações semânticas, o word2vec aproveita as *redes neurais*.

- Essas redes consistem em camadas interconectadas de nós que processam informações.

- Conforme ilustrado ao lado, as redes neurais podem ter muitas camadas onde cada conexão tem um certo peso, dependendo da entrada.

- Esses pesos são frequentemente referidos como os *parâmetros* do modelo.

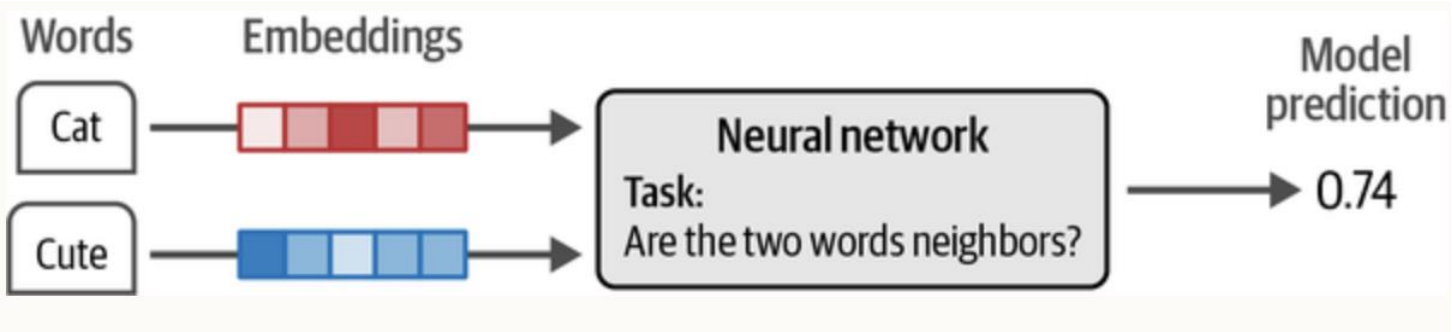


igura 1-6. Uma rede neural consiste em camadas interconectadas de nós onde cada conexão é uma equação linear.

Tokens e Embeddings

- Usando essas redes neurais, word2vec gera incorporações de palavras observando quais outras palavras elas tendem a aparecer ao lado de uma determinada frase.

- Começamos atribuindo todas as palavras em nosso vocabulário com uma incorporação de vetores, digamos, 50 valores para cada palavra inicializada com valores aleatórios.
- Então, em cada etapa de treinamento, conforme ilustrado, pegamos pares de palavras dos dados de treinamento e um modelo tenta prever se eles provavelmente serão ou não vizinhos em uma frase.



Tokens e Embeddings

- Durante este processo de treinamento, o word2vec aprende a relação entre as palavras e destila essa informação no *embedding*.

- Se as duas palavras tendem a ter os mesmos vizinhos, seus embeddings serão mais próximos uma da outra e vice-versa.

Tokens e Embeddings

- Os embeddings resultantes capturam o significado das palavras, mas o que exatamente isso significa?

- Para ilustrar esse fenômeno, vamos simplificar um pouco demais e imaginar que temos embeddings de várias palavras, a saber, “maçã” e “bebê”.

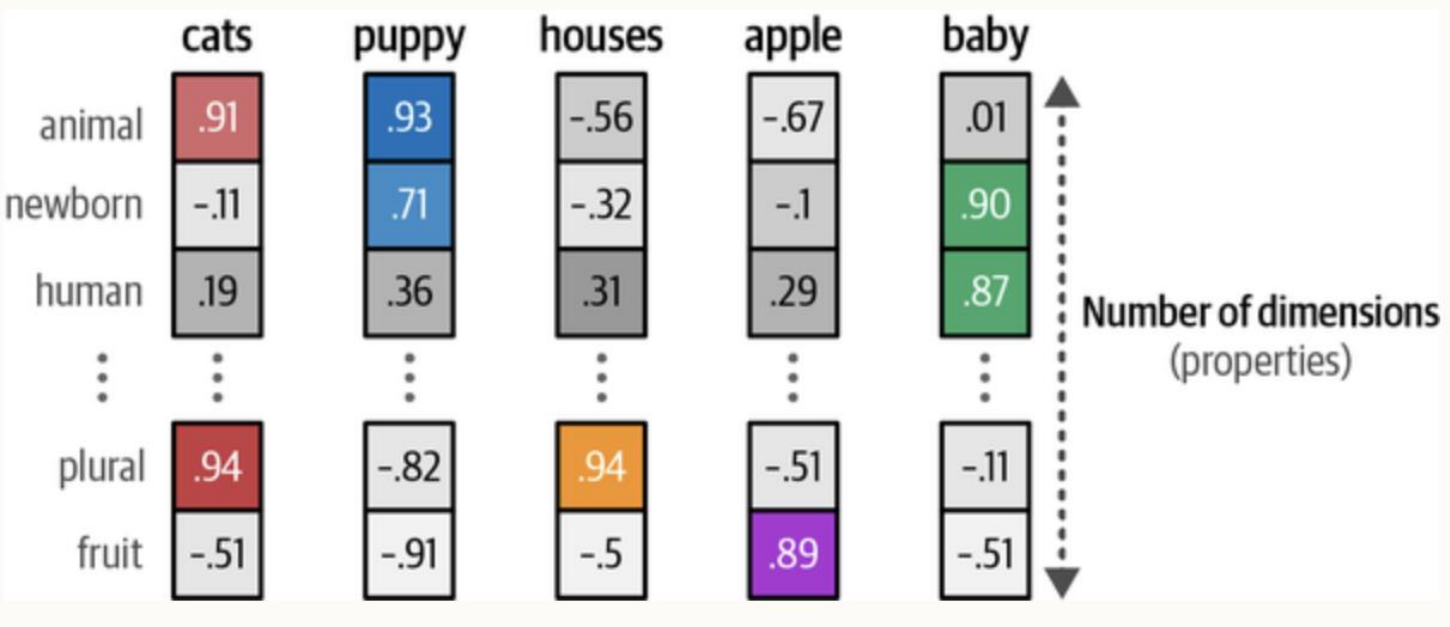
- Os embeddings tentam capturar o significado representando as propriedades das palavras.

- Por exemplo, a palavra “bebê” pode pontuar alto nas propriedades “recém-nascido” e “humano”, enquanto a palavra “maçã” pontua baixo nessas propriedades.

Tokens e Embeddings

- Conforme ilustrado ao lado, os embeddings podem ter muitas propriedades para representar o significado de uma palavra.

- Como o tamanho dos embeddings é fixo, suas propriedades são escolhidas para criar uma representação mental da palavra.



Tokens e Embeddings

- Na prática, essas propriedades geralmente são bastante obscuras e raramente se relacionam com uma única entidade ou conceito humanamente identificável.

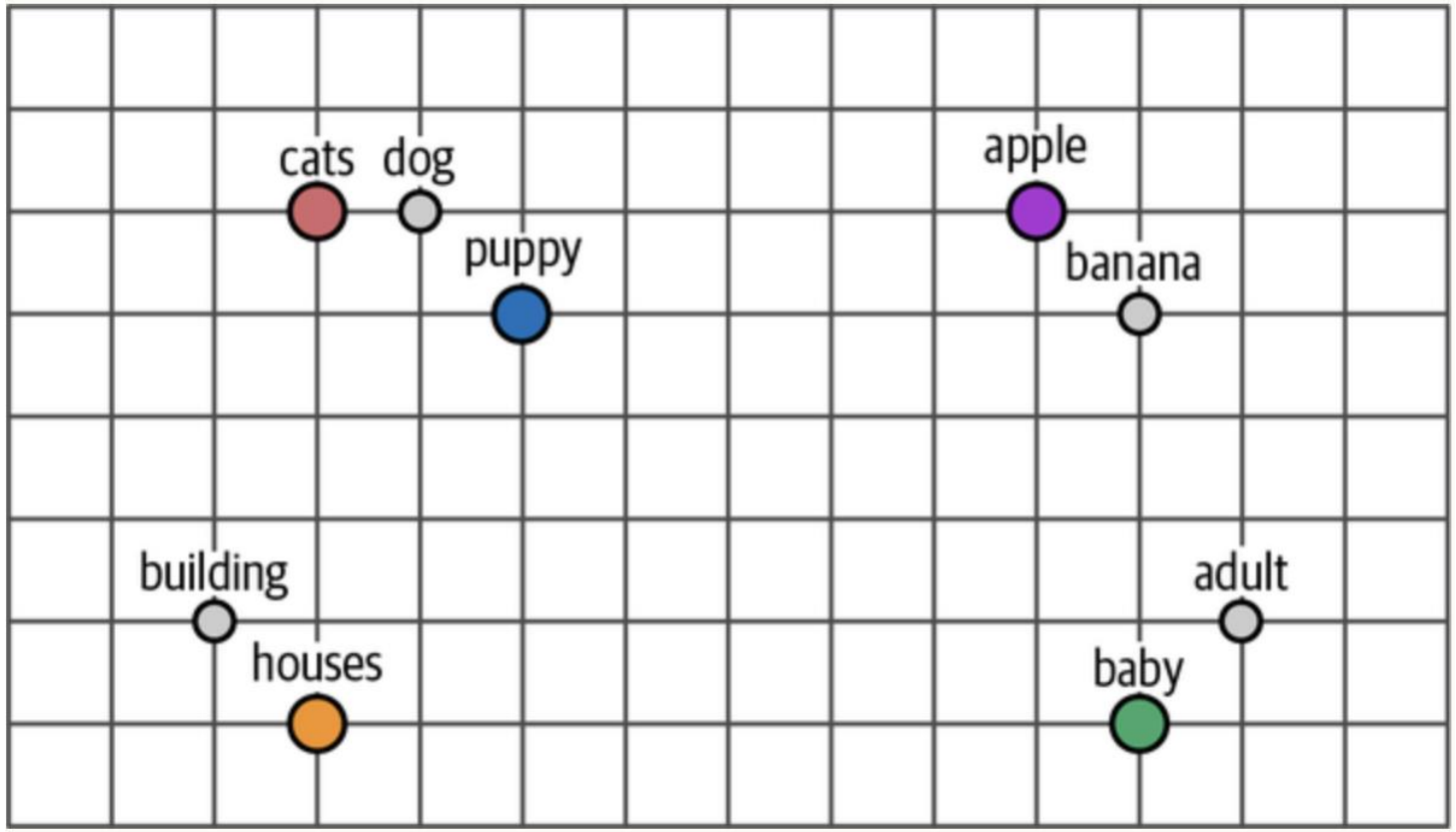
- No entanto, juntas, essas propriedades fazem sentido para um computador e servem como uma boa maneira de traduzir a linguagem humana para a linguagem do computador.

Tokens e Embeddings

- Os embeddings são tremendamente úteis, pois nos permitem medir a semelhança semântica entre duas palavras.

- Usando várias métricas de distância, podemos julgar o quão próxima uma palavra está de outra.

- Conforme ilustrado, se comprimissemos essess embeddings em uma representação bidimensional, você notaria que palavras com significado semelhante tendem a ser mais próximas.



Tokens e Embeddings

- Existem muitos tipos de embeddings, como embeddings de palavras e embeddings de frases que são usadas para indicar diferentes níveis de abstrações (palavra versus frase)

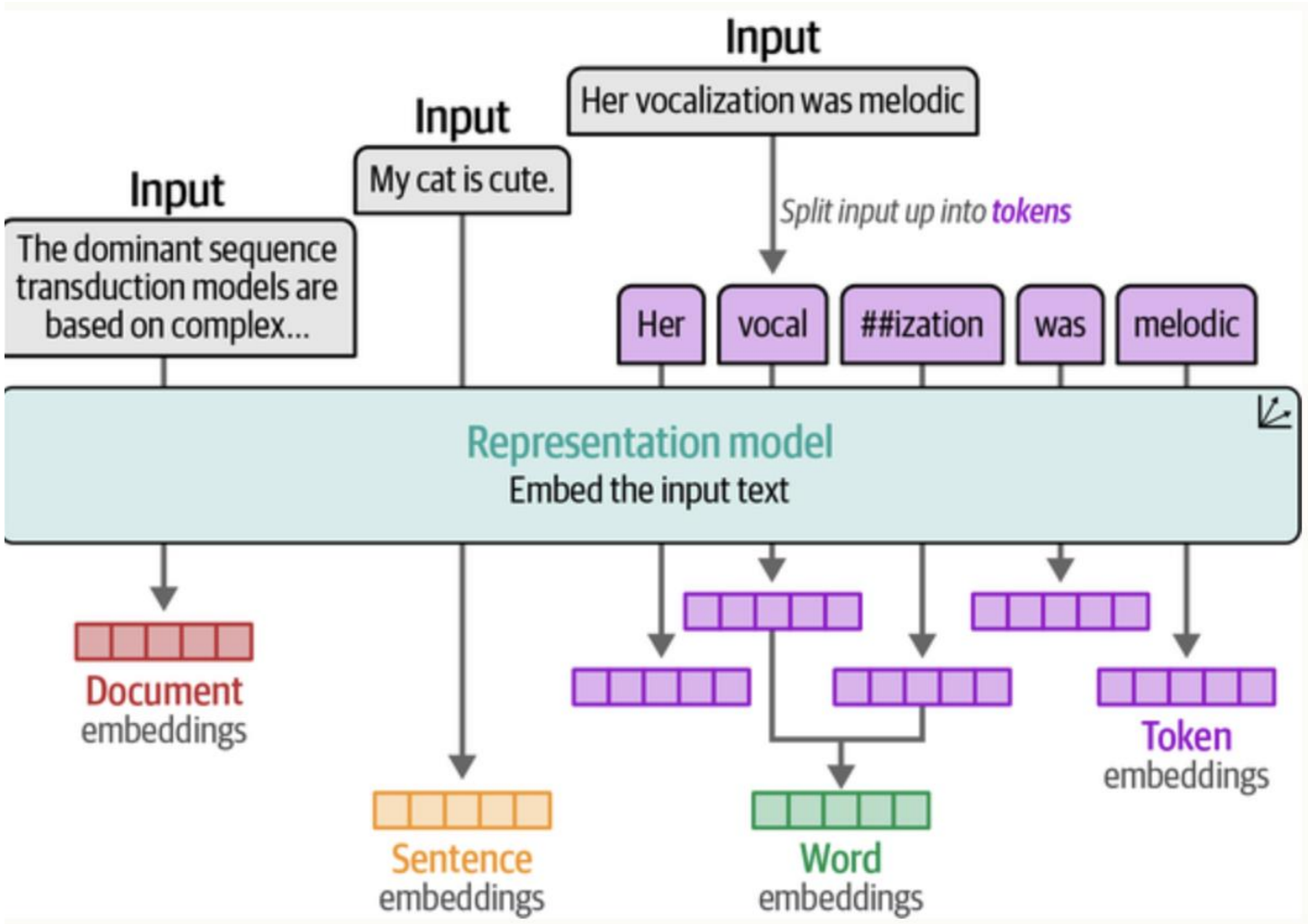


Figure 1-10. Embeddings can be created for different types of input.

Codificação e Decodificação de Contexto com Atenção

- O processo de treinamento do word2vec cria representações estáticas e para download de palavras.

- Por exemplo, a palavra “banco” sempre terá a mesma incorporação, independentemente do contexto em que é usada.

- No entanto, “banco” pode se referir tanto a um banco financeiro quanto à margem de um rio. Seu significado e, portanto, suas incorporações, devem mudar dependendo do contexto.

Codificação e Decodificação de Contexto com Atenção

Um passo na codificação deste texto foi alcançado através de redes neurais recorrentes (RNNs).

Estas são variantes de redes neurais que podem modelar sequências como uma entrada adicional.

Para fazer isso, esses RNNs são usados para duas tarefas:

- *codificando* ou representando uma frase de entrada e
- *decodificando* ou gerando uma frase de saída.

Codificação e Decodificação de Contexto com Atenção

- A figura ao lado ilustra esse processo mostrando como a frase “Eu amo Lhamas” é traduzido para o holandês;

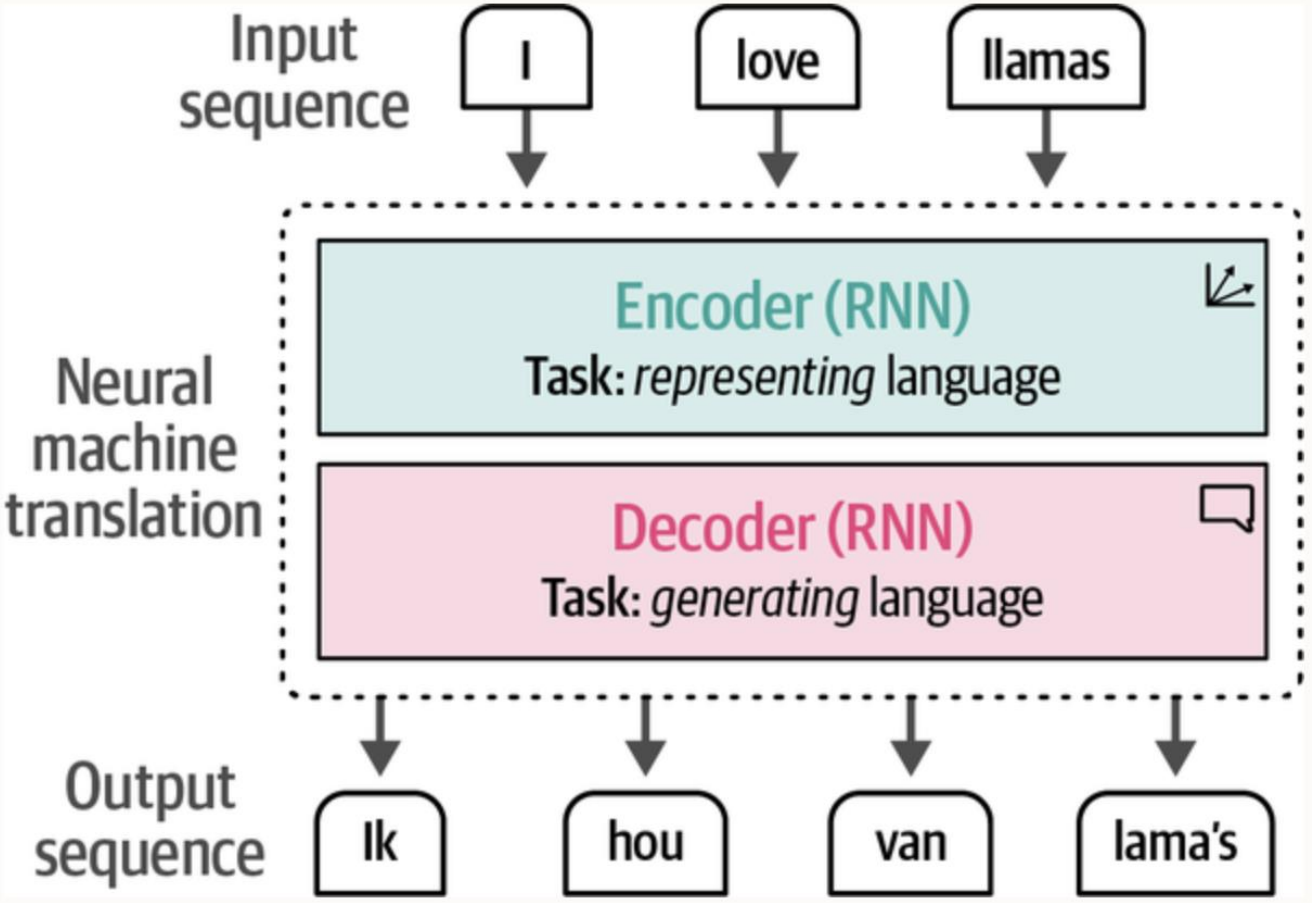


Figure 1-11. Two recurrent neural networks (decoder and encoder) translating an input sequence from English to Dutch.

Codificação e Decodificação de Contexto com Atenção

- Cada etapa nesta arquitetura é *autorregressiva*. Ao gerar a próxima palavra, essa arquitetura precisa consumir todas as palavras geradas anteriormente, como mostrado ao lado.

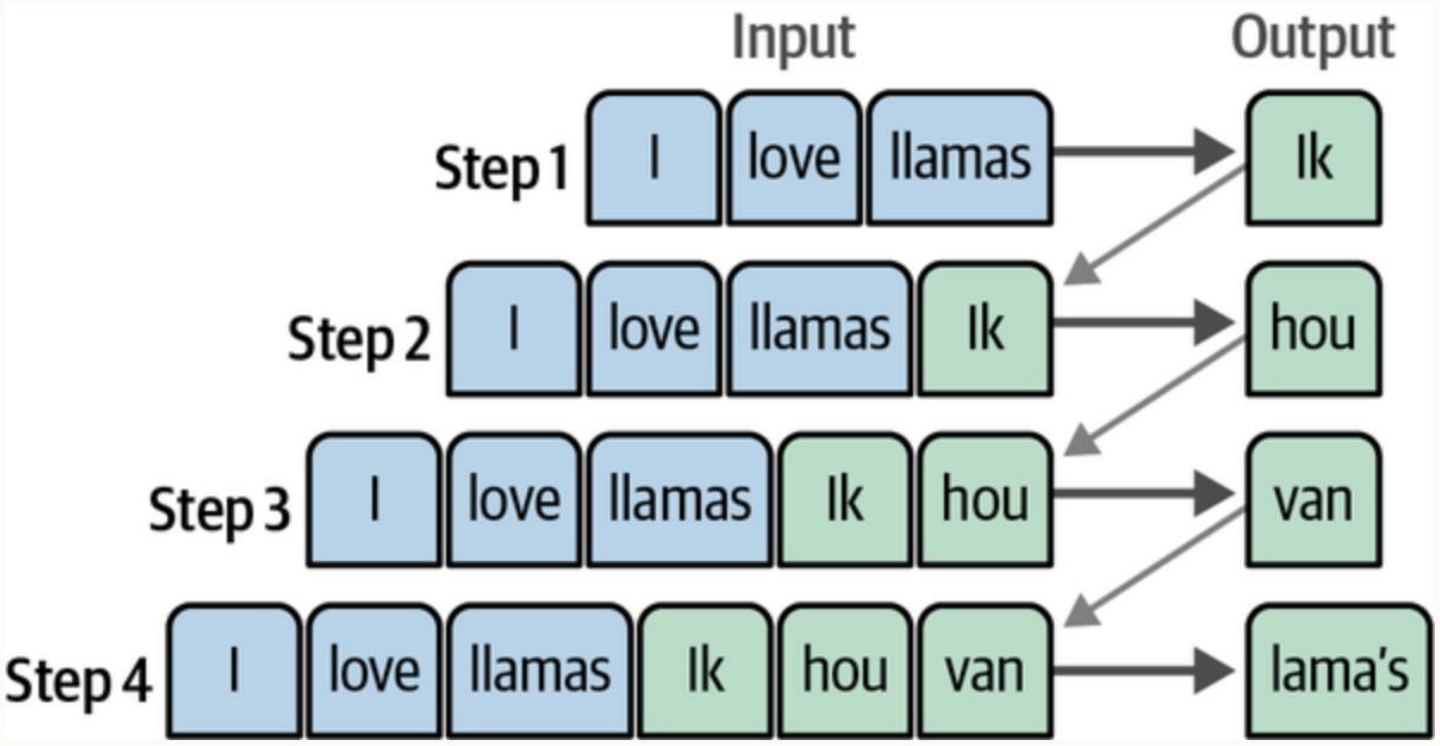


Figure 1-12. Each previous output token is used as input to generate the next token.

Codificação e Decodificação de Contexto com Atenção

- A etapa de codificação visa representar a entrada da melhor maneira possível, gerando o contexto na forma de uma incorporação, que serve como entrada para o decodificador.

- Para gerar essa representação, ele usa incorporações como suas entradas para palavras, o que significa que podemos usar word2vec para as representações iniciais.

- Na figura ao lado, podemos observar esse processo. Observe como as entradas são processadas sequencialmente, uma de cada vez, assim como a saída.

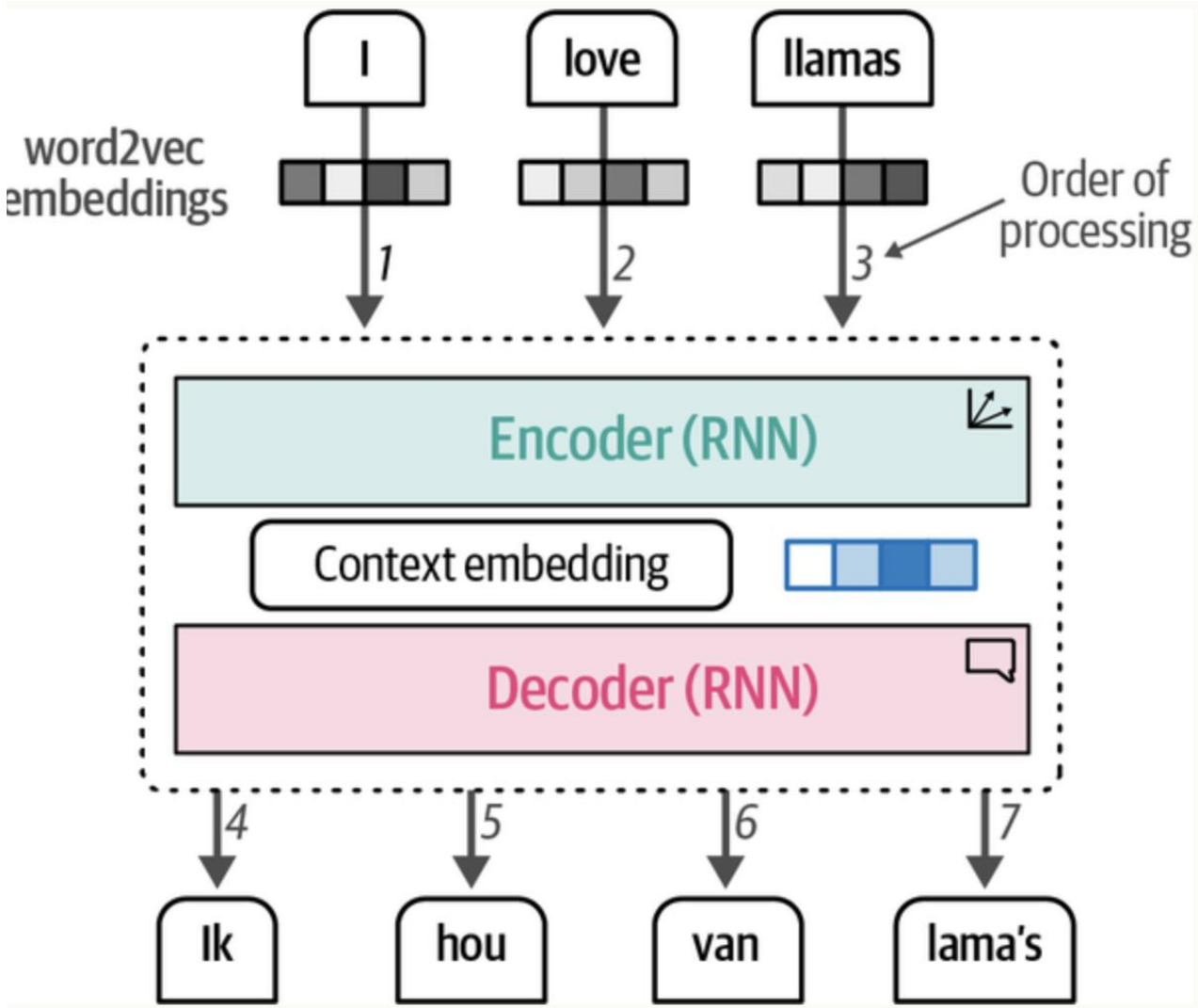


Figure 1-13. Using word2vec embeddings, a context embedding is generated that represents the entire sequence.

Codificação e Decodificação de Contexto com Atenção

- Essa incorporação de contexto, no entanto, dificulta o lidado com frases mais longas, uma vez que é apenas uma única incorporação que representa toda a entrada.

- Em 2014, foi introduzida uma solução chamada *atenção* que melhorou muito a arquitetura original.

- A atenção permite que um modelo se concentre em partes da sequência de entrada que são relevantes umas para as outras ("atender" umas às outras) e amplifique seu sinal, como mostrado ao lado.

- A atenção determina seletivamente quais palavras são mais importantes em uma determinada frase.

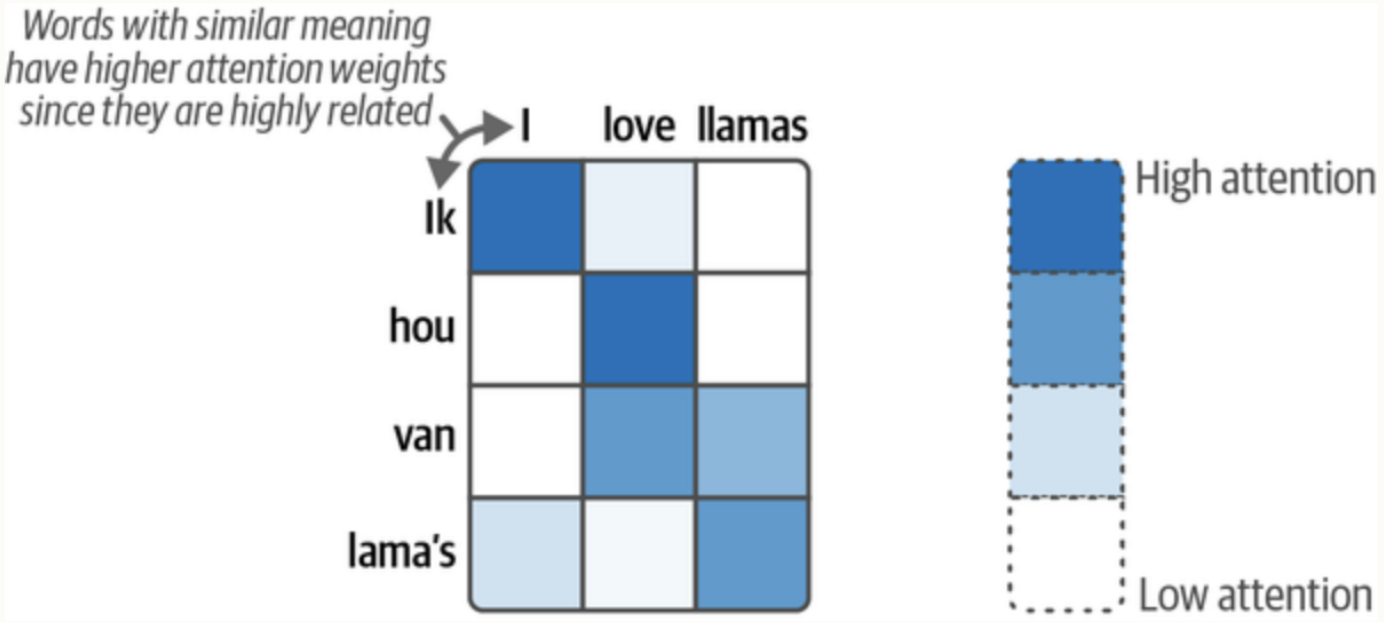


Figure 1-14. Attention allows a model to "attend" to certain parts of sequences that might relate more or less to one another.

Codificação e Decodificação de Contexto com Atenção

- Ao adicionar esses mecanismos de atenção à etapa do decodificador, o RNN pode gerar sinais para cada palavra de entrada na sequência relacionada à saída potencial.

- Em vez de passar apenas uma incorporação de contexto para o decodificador, os estados ocultos de todas as palavras de entrada são passados.

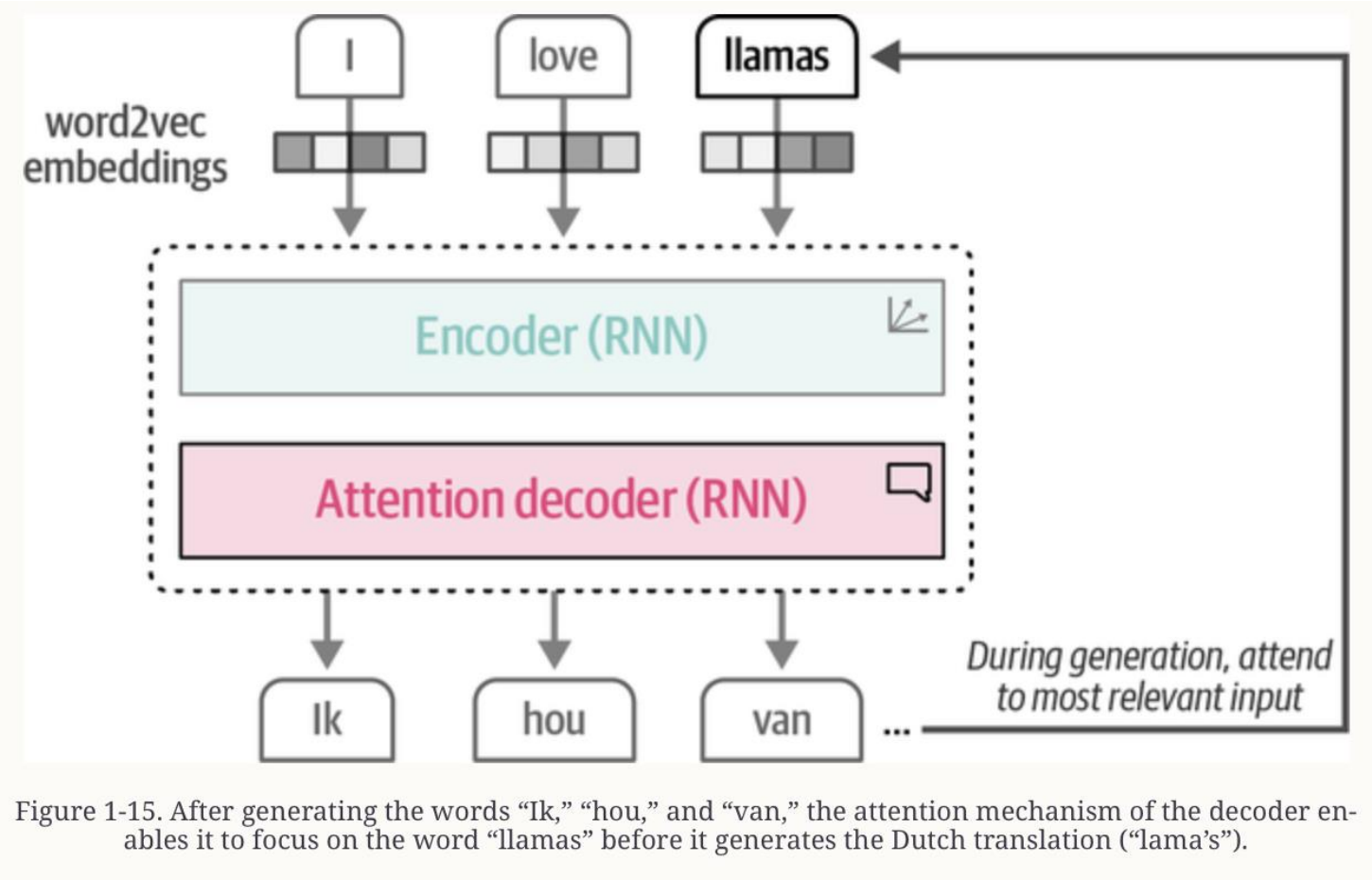


Figure 1-15. After generating the words “Ik,” “hou,” and “van,” the attention mechanism of the decoder enables it to focus on the word “llamas” before it generates the Dutch translation (“lama’s”).

Codificação e Decodificação de Contexto com Atenção

- Como resultado, durante a geração de “Ik hou van lama's”, o RNN acompanha as palavras que atende principalmente para realizar a tradução.

- Comparado ao word2vec, essa arquitetura permite representar a natureza sequencial do texto e o contexto em que ele aparece, “atendendo” toda a frase.

- Essa natureza sequencial, no entanto, impede a paralelização durante o treinamento do modelo.

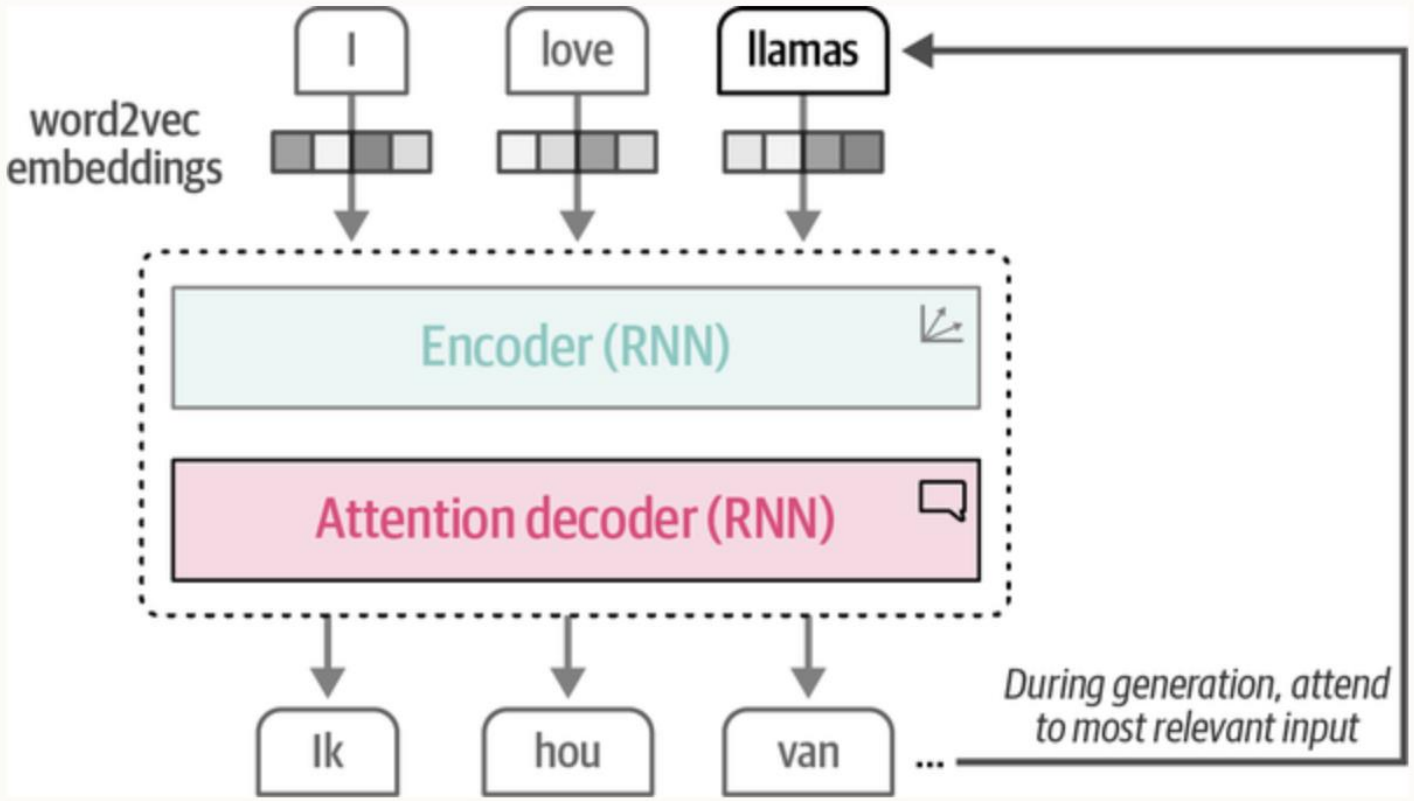


Figure 1-15. After generating the words “Ik,” “hou,” and “van,” the attention mechanism of the decoder enables it to focus on the word “llamas” before it generates the Dutch translation (“lama’s”).

Attention is all you need

- O verdadeiro poder da atenção, e o que impulsiona as incríveis habilidades de grandes modelos de linguagem, foi explorado pela primeira vez no conhecido artigo "Attention is all you need" lançado em 2017.

- Os autores propuseram uma arquitetura de rede chamada theTransformer, que era baseada exclusivamente no mecanismo de atenção e removeu a rede de recorrência que vimos anteriormente.

- Comparado à rede de recorrência, o Transformador poderia ser treinado em paralelo, o que acelerou tremendamente o treinamento.

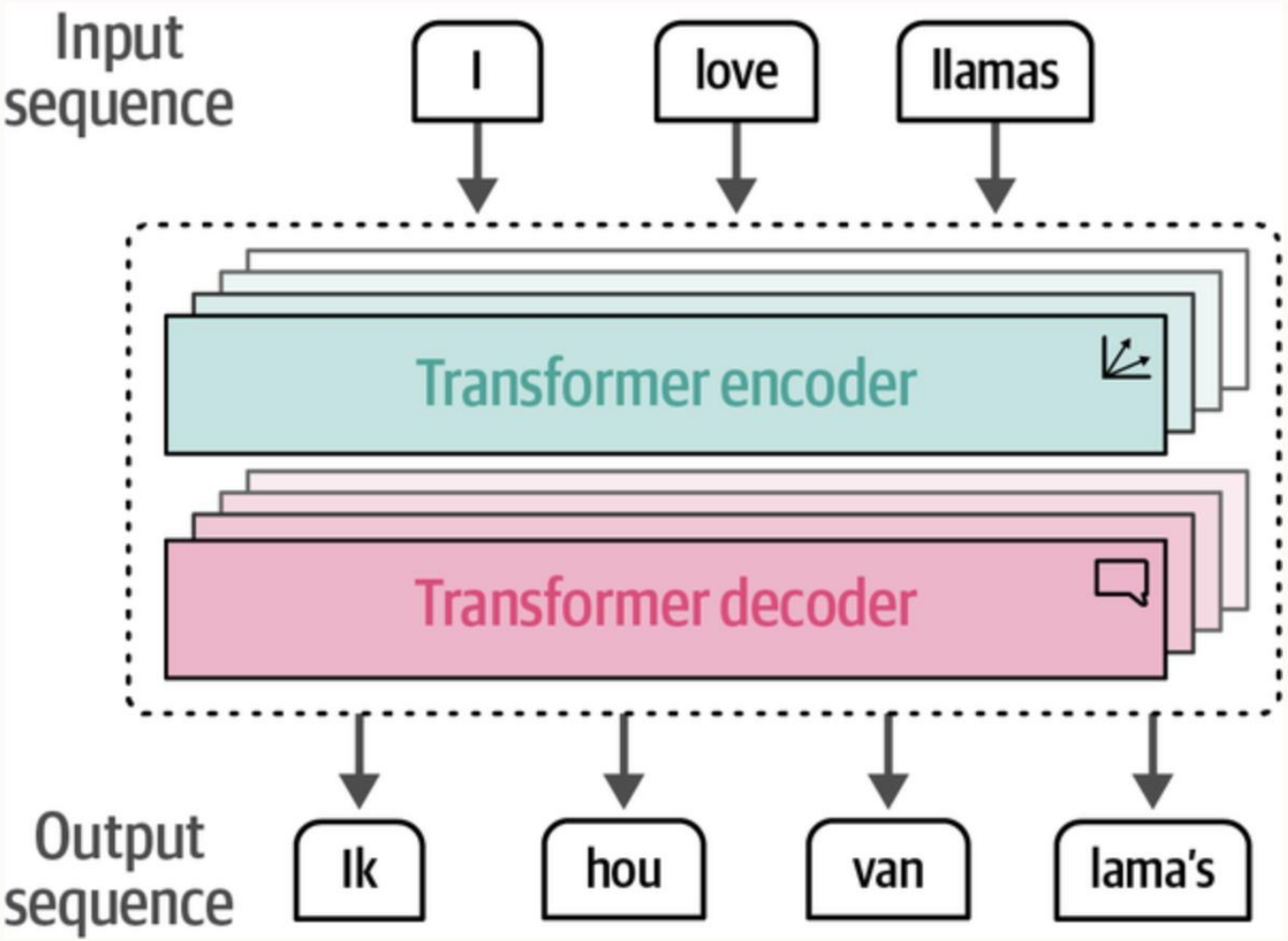


Figure 1-16. The Transformer is a combination of stacked encoder and decoder blocks where the input flows through each encoder and decoder.

Attention is all you need

- No Transformer, os componentes de codificação e decodificador são empilhados um em cima do outro, conforme ilustrado ao lado.

- Essa arquitetura permanece autorregressiva, precisando consumir cada palavra gerada antes de criar uma nova palavra.

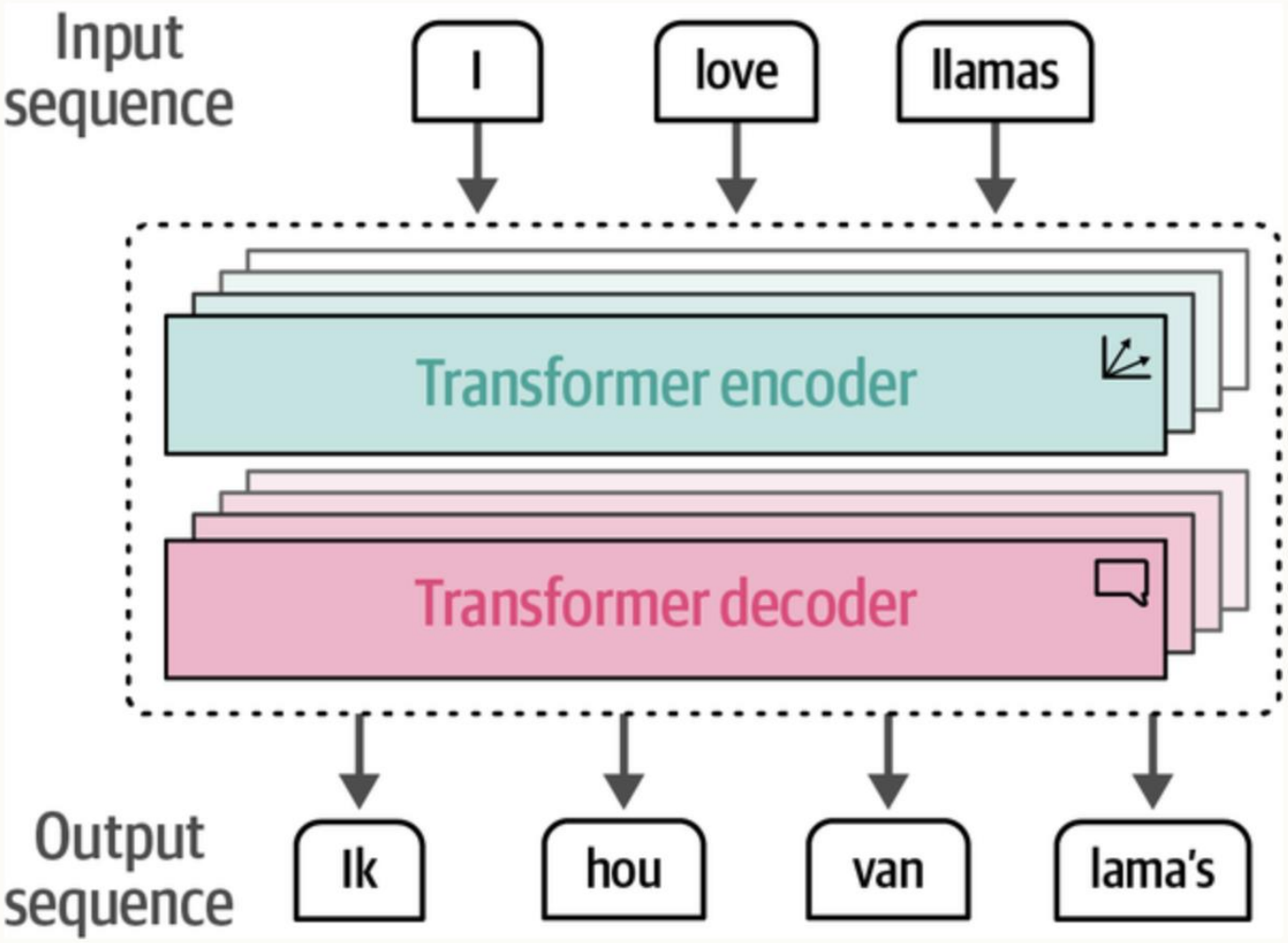


Figure 1-16. The Transformer is a combination of stacked encoder and decoder blocks where the input flows through each encoder and decoder.

Attention is all you need

- Agora, tanto os blocos do codificador quanto o decodificador girariam em torno da atenção em vez de alavancar um RNN com recursos de atenção.

- O bloco do codificador no Transformador consiste em duas partes, *auto-atenção* e uma *rede neural feedforward*, que são mostradas na figura.

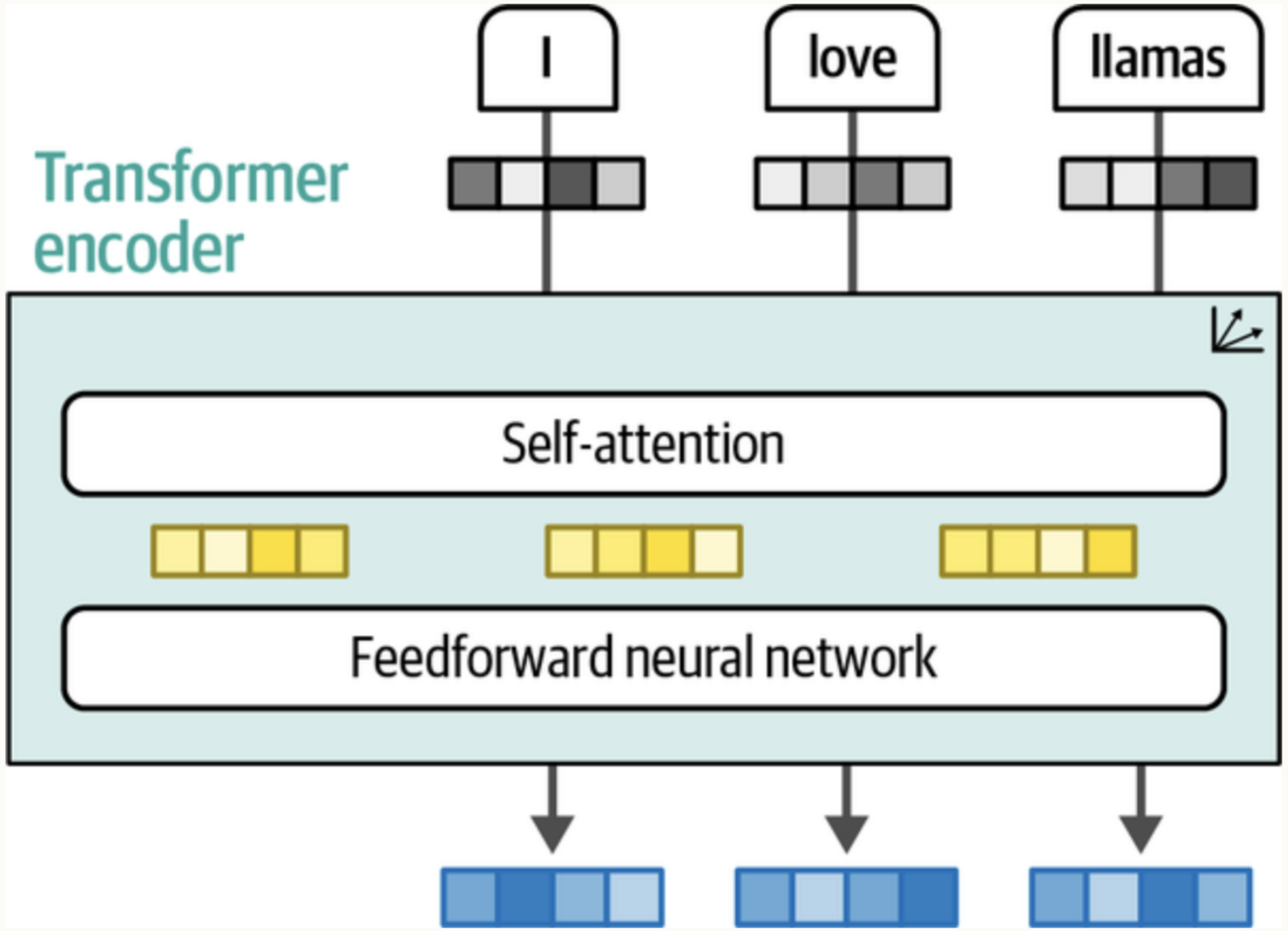


Figure 1-17. An encoder block revolves around self-attention to generate intermediate representations.

Attention is all you need

- Em comparação com os métodos anteriores de atenção, a auto-atenção pode atender a diferentes posições dentro de uma única sequência, representando assim com mais facilidade e precisão a sequência de entrada, conforme ilustrado ao lado.

- Em vez de processar um token de cada vez, ele pode ser usado para analisar toda a sequência de uma só vez.

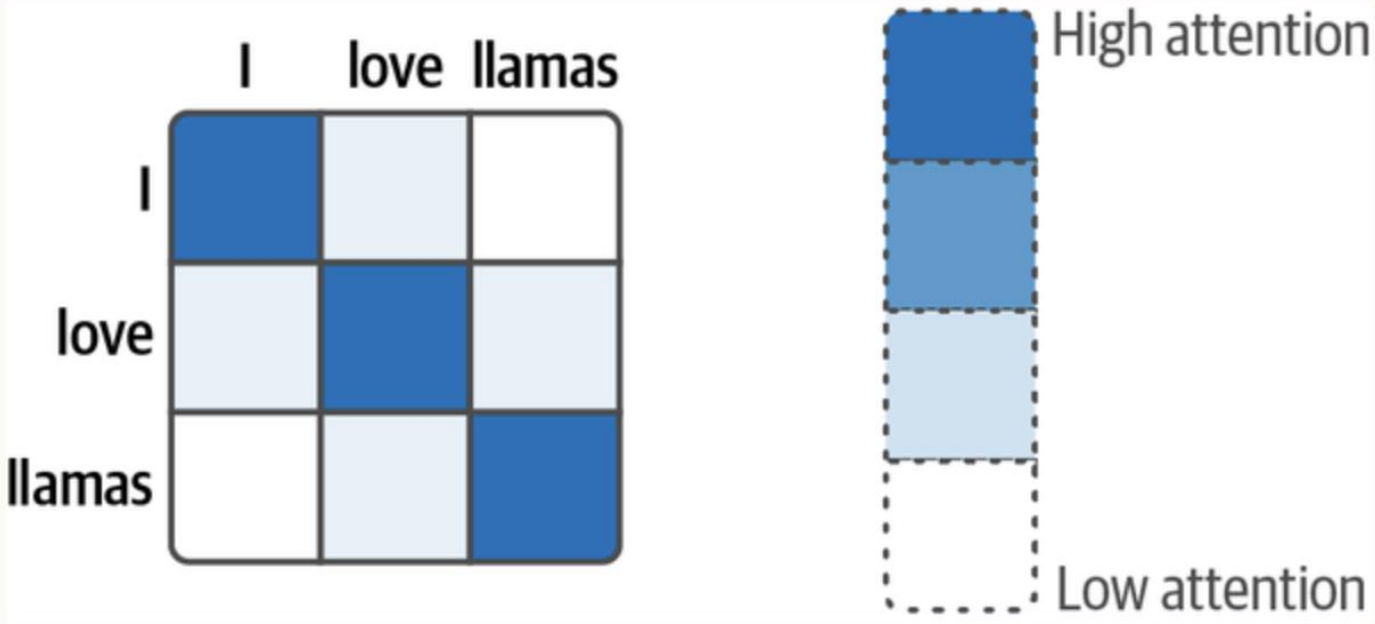


Figure 1-18. Self-attention attends to all parts of the input sequence so that it can “look” both forward and back in a single sequence.

Attention is all you need

- Comparado ao codificador, o decodificador tem uma camada adicional que presta atenção à saída do codificador (para encontrar as partes relevantes da entrada).

- Conforme demonstrado na figura, esse processo é semelhante ao decodificador de atenção RNN que discutimos anteriormente.

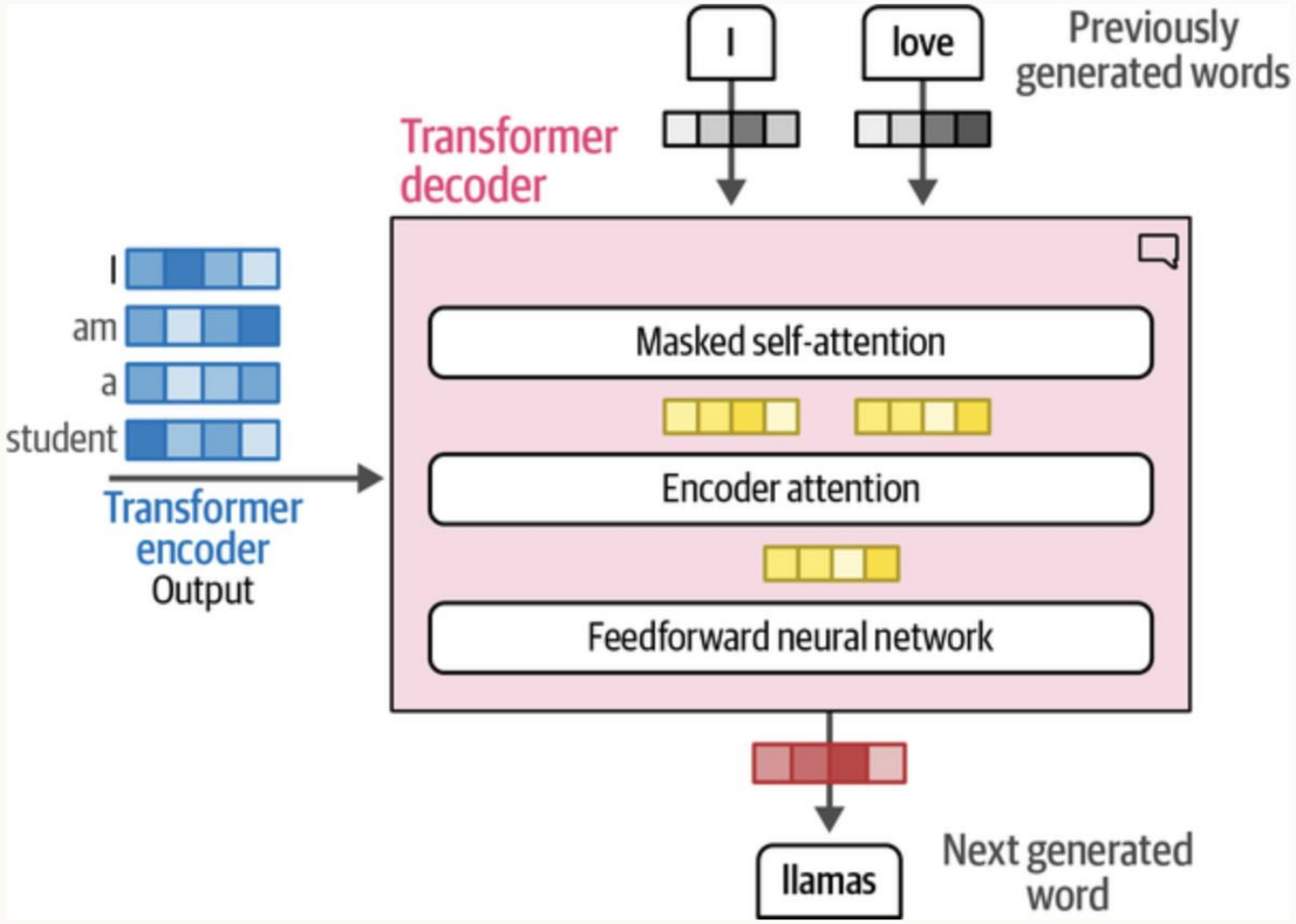


Figure 1-19. The decoder has an additional attention layer that attends to the output of the encoder.

Attention is all you need

- Como mostrado na figura, a camada de auto-atenção no decodificador mascara posições futuras para que atenda apenas a posições anteriores para evitar vazamento de informações ao gerar a saída.

- Juntos, esses blocos de construção criam a arquitetura Transformer e são a base de muitos modelos impactantes em IA de Linguagem, como BERT e GPT-1.

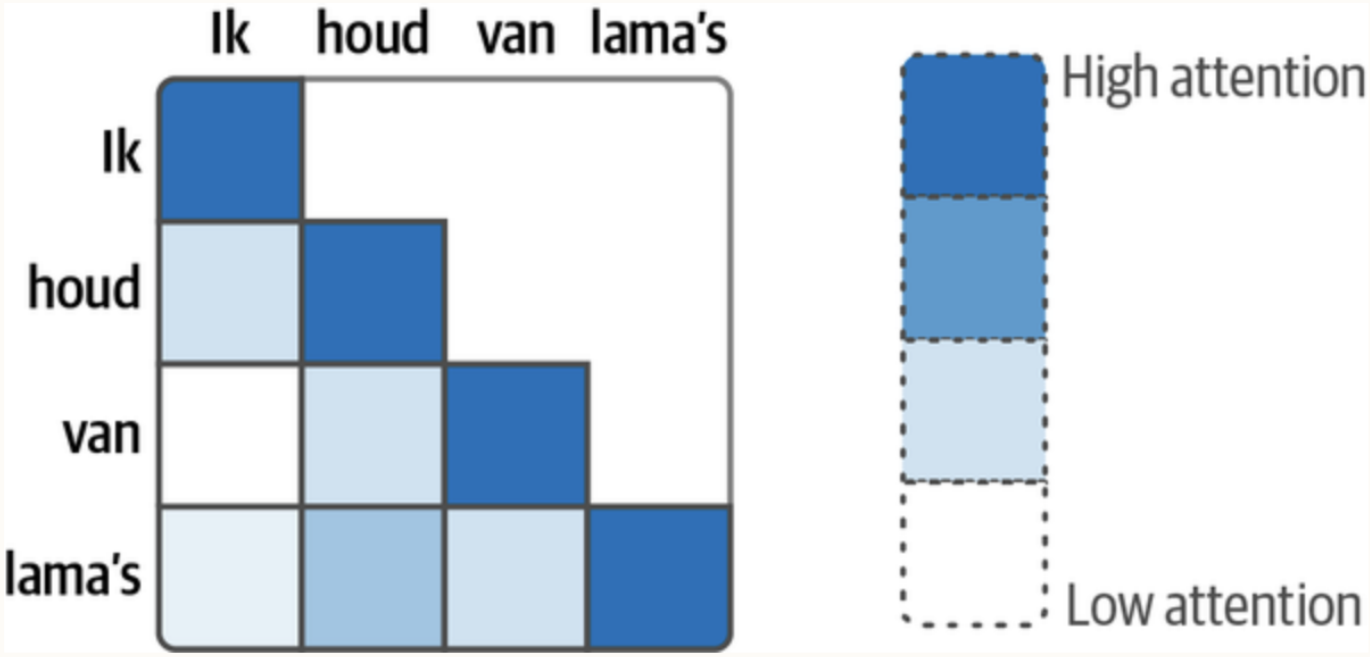


Figure 1-20. Only attend to previous tokens to prevent "looking into the future."